Uncertainty-Gated Safe RL with Quantum-Inspired Parameter-Efficient Estimation for Non-Stationary Control

SHIVANG GUPTA
Indian Institute of Technology Roorkee
shivang_g@ar.iitr.ac.in

Abstract

Safe RL breaks in a particular way under regime shift: the first symptom is often not a lower mean return, but a controller that becomes jumpy and hard to predict when its own estimates are noisy. This matters in governance-like feedback loops where the operational question is ''what happens on the bad days'' (bursts, attacks, latency spikes), not only what happens on average.

We evaluate a decision-time safety layer that sits on top of PPO: a calibrated uncertainty gate chooses between a learned action and a deterministic fallback, and a deterministic shield enforces hard constraints before execution. The key claim is architectural. The combination (gate + shield) can move the safety--stability operating point under shift relative to policy-only execution, and a compact, classically simulated VQC-style head provides a parameter-efficient uncertainty signal for that routing decision.

Evidence comes from two tiers: (i) standard constrained benchmarks (SafetyPointGoal1-v0 and SafetyHalfCheetahVelocity-v1) with a fixed 30-seed protocol, and (ii) matched 50-seed governance-controller comparisons used as stress tests. To keep the story honest, we also surface a negative result: in one stylized governance ablation, the full system's reward is worse than random. We keep it because it delineates scope: this method targets stable, safe behavior under shift and tight parameter budgets; it is not a guarantee of reward dominance under every reward design.

Repository artifacts and scripts are available upon request.

1 Introduction

Two themes motivate QAI-Chain. First, controllers deployed in governance-like loops fail under shift in ways that mean-return plots conceal: the policy may keep a reasonable average reward while producing brittle, high-variance actions when telemetry is adversarial or the regime changes mid-episode. Second, in those regimes it is common to keep a deterministic rule-based controller in the loop anyway; the engineering question is how to integrate learning with that reality cleanly.

We study a concrete pattern: *uncertainty-gated execution with deterministic shielding.* PPO proposes an action, an uncertainty estimate decides whether to execute it or fall back to a deterministic heuristic, and a shield enforces hard bounds and rate limits before the environment sees the control. The ''quantum-inspired'' element is used as an efficiency mechanism: a compact, classically simulated VQC-style head provides a nonlinear uncertainty signal at low parameter count. In a hard regime we observe higher fallback precision at matched trigger rate (intensity 1.0: 0.964 vs 0.743 for MC-dropout).

Scope is intentionally narrow. The claim is not a quantum advantage statement and not a promise of universal reward dominance. The claim is architectural: under rapid switching, routing by

calibrated uncertainty plus a deterministic shield yields a more stable safety profile than policy-only execution, and can be implemented with a compact estimator instead of a large ensemble.

Empirically we anchor external validity on standard constrained benchmarks (SafetyPointGoal1-v0 and SafetyHalfCheetahVelocity-v1) under a fixed 30-seed protocol, then treat governance simulations as stress tests with matched 50-seed comparisons. We also keep ''honest baselines'' in the loop: deterministic heuristics are not straw men, and in easier regimes they remain strong.

Implementation-wise, we change as little as possible relative to PPO: (i) variance-aware reward shaping during training, and (ii) a decision-time routing rule controlled by a threshold τ and a risk weight λ_{risk}. Across complexity levels we keep the model class and environment fixed and retune only these few scalars, so comparisons track the effect of the safety layer rather than a shifting architecture.

On-chain audit commitments appear as an optional deployment layer for provenance and incident review; they are orthogonal to the core safe-control mechanism evaluated in this manuscript.

Contributions. The paper makes the following contributions:

- a decision-time safe-control architecture that combines uncertainty-gated fallback with deterministic shielding,
- a parameter-efficient uncertainty-estimation module (compact architecture) that preserves safety behavior under non-stationary disturbances,
- and a high-rigor evaluation protocol centered on completed 30-seed standard constrained-benchmark runs (SafetyPointGoal1-v0 and SafetyHalfCheetahVelocity-v1), with a matched 50-seed governance controller-comparison suite and governance stress tests as supporting evidence.

Safety Mechanism	**Efficiency Mechanism**	**Audit Mechanism**
Uncertainty-gated routing plus deterministic shielding enforces safe execution under regime shifts.	A compact uncertainty head reduces parameter count while preserving control quality.	Optional tamper-evident commitments support governance traceability.
Primary algorithmic claim.	Secondary efficiency mechanism.	Deployment layer; not required for core Safe RL results.

Table 1: Core contribution layering: what is algorithmically novel versus efficiency support versus deployment engineering.

Scope. The empirical claim is specific and testable: uncertainty-gated control with compact architecture achieves comparable safety behavior at materially lower parameter budget under non-stationary governance dynamics, and this can be measured with fixed-seed paired comparisons [3, 10, 12].

2 Related Perspectives

Blockchain and ML integration work often focuses on transaction analytics, mechanism optimization, or fraud detection in mostly stationary settings. Governance control with adversarial perturbations has received less systematic, reproducible treatment. Classical consensus foundations remain

essential [7], and PPO remains a practical baseline for continuous-control policy learning [11], but both need stress-tested evaluation when used in operational control loops.

Safe RL literature offers several tools with different assumptions: CVaR objectives [2], robust MDP formulations [6], and constrained policy optimization [1]. Recent practice also emphasizes distribution-shift robustness, risk-sensitive policy updates, and uncertainty-aware action filtering. The approach here keeps PPO as the optimizer and adds uncertainty-aware routing at action time with risk shaping during updates. We evaluate this controller pattern against strong heuristic and constrained baselines under the same disturbance model, and we explicitly treat this paper as a systems-and-control contribution rather than a new policy-optimization objective.

Relative to modern safe-RL directions (risk-sensitive updates, uncertainty-aware filtering, and robust constrained control), this work contributes primarily at the control-architecture layer: it specifies how to combine branch-level uncertainty routing with deterministic post-check execution in a reproducible deployment pipeline. Expanding direct head-to-head comparison against additional recent baselines is an explicit next step, not an omitted assumption.

Reproducibility details are provided in dedicated checklist material, while the main text focuses on algorithmic behavior and safety-relevant outcomes.

3 System Design

The main system claim is a safe RL control loop with compact uncertainty estimation, while blockchain serves as a deployment-time audit substrate. For clarity, the online decision path is:

$$\text{state telemetry} \rightarrow \text{policy proposal} \rightarrow \text{uncertainty gate} \rightarrow \text{shield} \rightarrow \text{executed action}.$$

Audit records are committed to chain as a verifiable log after execution. This division keeps the core method in the RL/safety domain while preserving multi-party provenance when control actions are reviewed by mutually untrusted participants.

3.1 System Diagram

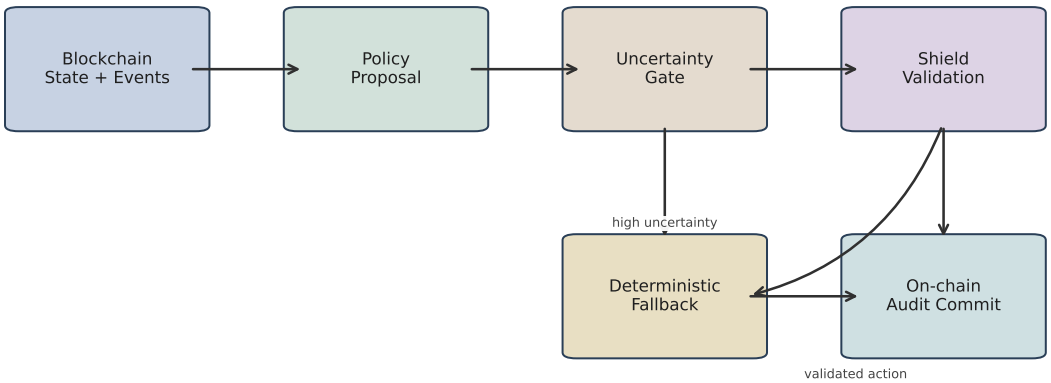

Figure 1: System flowchart

3.2 Data and Control Flow

Transaction and peer events update chain state through RPC and validation logic. Governance telemetry is encoded into a compact vector, and the controller emits bounded actions for difficulty, block size, and gas limit:

$$\begin{aligned}\text{State telemetry} &\rightarrow \text{policy proposal} \rightarrow \text{uncertainty gate} \\ &\rightarrow \text{safety shield} \rightarrow \text{on-chain commit.}\end{aligned} \tag{1}$$

The uncertainty gate can route to deterministic fallback before execution; the shield can reject or clamp invalid actions. This paper's performance claims are tied to that gated-and-shielded control path. Extended blockchain architecture detail is provided in the appendix.

3.3 API Contracts

RPC endpoints use typed request/response schemas. Invalid payloads are rejected and negative-path contract tests cover malformed or missing fields.

Contract tests are the boundary between protocol logic and adaptive governance logic.

3.4 Operational Guardrails

Deployment reliability depends on typed interfaces, contract-tested RPC boundaries, and deterministic control handoff between policy, gate, and shield. These controls reduce interface-level failure modes and keep execution behavior auditable across nodes.

4 Methodology

4.1 Control Objective

The governance objective combines throughput, latency, decentralization proxy, and target tracking:

$$\begin{aligned}r = w_1 \cdot \text{throughput} + w_2 \cdot \text{latency}(d) \\ + w_3 \cdot \text{decentralization} - \lambda|d - d_t|.\end{aligned}$$

Action outputs are bounded and then validated by a safety shield before execution.

4.2 Policy Family

We evaluate random, heuristic, PPO-family, and robust gated controllers. PPO uses clipped updates [11]. The robust branch adds risk shaping:

$$\tilde{r}_t = r_t - \lambda_{\text{risk}} \operatorname{Var}(r_{t-k:t}).$$

4.3 Shield Logic (Executable Rule)

The shield is deterministic and state-aware. At each step, it enforces bounded actions and bounded action-rate change before execution:

$$a_t^{\text{exec}} = \text{clip}(a_t, a_{\min}, a_{\max}),$$

$$\Delta a_t = \text{clip}(a_t^{\text{exec}} - a_{t-1}, -\delta_{\max}, \delta_{\max}).$$

If a post-check still violates a hard safety predicate (for example, projected delay or partition threshold), the shield routes to deterministic fallback a_t^{heur} and re-validates. In implementation terms:

- If `hard_safe(state, action)` is false, clamp action by bounds and max delta.
- Re-check safety; if still false, switch to `heuristic_action(state)`.
- Return the validated action.

For explicit auditability, shield execution is summarized below:

1. Input (s_t, a_t, a_{t-1}).
2. Set $a \leftarrow \text{clip}(a_t, a_{\min}, a_{\max})$.
3. Set $a \leftarrow a_{t-1} + \text{clip}(a - a_{t-1}, -\delta_{\max}, \delta_{\max})$.
4. If hard_safe(s_t, a), return a.
5. Set $a_h \leftarrow$ heuristic_action(s_t).
6. Set $a_h \leftarrow \text{clip}(a_h, a_{\min}, a_{\max})$.
7. Set $a_h \leftarrow a_{t-1} + \text{clip}(a_h - a_{t-1}, -\delta_{\max}, \delta_{\max})$.
8. If hard_safe(s_t, a_h), return a_h.
9. Otherwise, return emergency_safe_action(s_t).

4.4 Quantum-Inspired Uncertainty Gate

The policy front-end is a 6-qubit variational circuit with angle embedding, entangling layers, data re-uploading, and Pauli-Z measurements for mean/scale signals. Uncertainty is estimated by parameter perturbation:

$$u_q(x) = \text{Var}\left(\{\mu_{\theta+\epsilon_k}(x)\}_{k=1}^{K}\right),$$
$$\epsilon_k \sim \mathcal{N}(0, 0.01I), \quad K = 10.$$

The routing rule is

$$a_t = \begin{cases} a_t^{\text{heur}}, & u_q(x_t) > \tau_q, \\ a_t^{\text{policy}}, & u_q(x_t) \leq \tau_q. \end{cases}$$

4.5 Engineering Trade-offs in Uncertainty Design

The uncertainty estimator was chosen with deployment latency in mind, not only offline accuracy. Deep ensembles improve diversity but require maintaining and serving multiple full policy copies per decision step. In contrast, parameter perturbation reuses one compact model and injects small weight noise at inference, which keeps implementation complexity and memory footprint lower in our control loop.

In head-to-head tests, this choice gave a practical balance: quantum parameter-perturbation uncertainty achieved higher precision at the same trigger budget than MC-dropout and ensemble baselines, while preserving comparable recall. We therefore prioritized estimator compactness plus

decision quality under fixed trigger-rate constraints over maximizing any single uncertainty score in isolation.

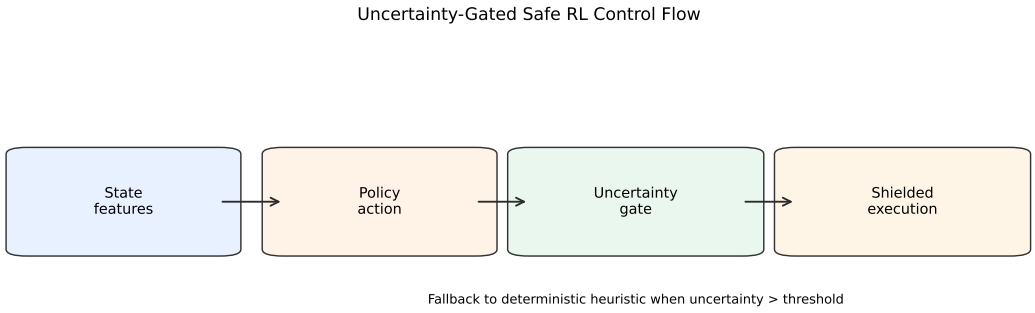

Figure 2: Quantum-inspired uncertainty-gated fallback and shielded action execution.

4.6 Why the Compact Estimator Compresses Well Here

The 6-qubit VQC-style module is classically simulated in this study, and the design priority is parameter efficiency rather than raw throughput. Governance states couple a few dominant factors (delay, congestion, adversarial pressure) with strong interaction effects. Entangling layers provide multiplicative feature interactions in a compact parameterization, and data re-uploading lets the same qubits express higher-order nonlinear responses over time. In this setting, that combination approximates the required control surface with far fewer trainable weights than a dense MLP. Empirically, compression is most effective when paired with uncertainty gating and shielding, which stabilize behavior during regime shifts.

4.7 Adversarial and Ablation Design

The scaled benchmark injects burst demand, delay noise, attack pressure, and regime shifts. We evaluate seven configurations:

- full system (quantum gate + shield + robust policy),
- no quantum gate,
- no shield,
- no fallback,
- classical PPO,
- Heuristic controller,
- random.

The stress-aware reward is

$$\begin{aligned} r_t^{\mathrm{adv}} = \alpha\,\mathrm{throughput}_t - \beta\,\mathrm{delay}_t - \gamma\,\mathrm{attack}_t \\ - \eta\,\mathrm{fork}_t - \lambda|d_t - d^\star|. \end{aligned}$$

4.8 Evaluation

Primary metrics are reward, safety violations, target MAE, fallback rate, and latency proxy. For base PPO-family comparisons we report bootstrap intervals, permutation tests, and Cohen's d [4,5].

5 Theory: Safety and Stability Guarantees

This section formalizes the mechanism-level guarantees of the proposed controller under explicit assumptions. The intent is to clarify why the architecture behaves as observed in simulation, not to claim assumption-free optimality theorems for all deployment settings.

Let s_t be the observed state, a_t^π the learned policy action, a_t^h the fallback heuristic action, and $u_q(s_t)$ the quantum-inspired uncertainty estimate. Let the switching rule be

$$\hat{a}_t = \begin{cases} a_t^h, & u_q(s_t) > \tau_q, \\ a_t^\pi, & u_q(s_t) \le \tau_q. \end{cases}$$

Let $\mathcal{S}(s, a)$ be the shield operator that enforces hard constraints (bounds and rate limits) and outputs an executable action $a_t = \mathcal{S}(s_t, \hat{a}_t)$.

Theorem 1 (Safety Bound with Shielded Quantum-Inspired Gating). Assume the shield is complete for hard constraints, i.e., for every state-action pair (s, a) the output $\mathcal{S}(s, a)$ satisfies all hard safety constraints. Then:

1. Within the simulation environment, and assuming a bug-free and complete shield implementation, the probability of catastrophic action is zero.[1]
2. Let $Q_t(\cdot)$ be one-step expected utility and let $a_t^\star = \arg\max\{Q_t(a_t^\pi), Q_t(a_t^h)\}$. If quantum-inspired uncertainty calibration error is bounded by $\epsilon_{\text{quantum}}$ and the shield induces utility deviation at most ϵ_{shield}, then

$$\mathbb{E}[Q_t(a_t^\star) - Q_t(a_t)] \le \epsilon_{\text{quantum}} + \epsilon_{\text{shield}}.$$

Proof. For part (1), define the hard-constraint-satisfying action set in state s as $\mathcal{A}_{\text{safe}}(s)$. Completeness of $\mathcal{S}$ states

$$\forall (s, a), \quad \mathcal{S}(s, a) \in \mathcal{A}_{\text{safe}}(s).$$

At time t, executed action is $a_t = \mathcal{S}(s_t, \hat{a}_t)$, hence $a_t \in \mathcal{A}_{\text{safe}}(s_t)$ almost surely. Catastrophic actions are exactly actions outside $\mathcal{A}_{\text{safe}}(s_t)$, therefore

$$\Pr(a_t \notin \mathcal{A}_{\text{safe}}(s_t)) = 0.$$

For part (2), write one-step regret

$$R_t := Q_t(a_t^\star) - Q_t(a_t) = Q_t(a_t^\star) - Q_t(\mathcal{S}(s_t, \hat{a}_t)).$$

Add and subtract $Q_t(\hat{a}_t)$:

$$R_t = \underbrace{Q_t(a_t^\star) - Q_t(\hat{a}_t)}_{\text{switching regret } R_t^{\text{sw}}} + \underbrace{Q_t(\hat{a}_t) - Q_t(\mathcal{S}(s_t, \hat{a}_t))}_{\text{shield regret } R_t^{\text{sh}}}.$$

[1] This is an in-simulator guarantee, not a real-world absolute guarantee. Deployment can fail due to software defects, model mismatch, sensing faults, or incomplete constraint encoding; therefore operational risk controls remain necessary.

By assumption, shield projection loss satisfies $R_t^{\text{sh}} \le \epsilon_{\text{shield}}$ almost surely, hence $\mathbb{E}[R_t^{\text{sh}}] \le \epsilon_{\text{shield}}$.

For the switching term, define $\Delta_t := |Q_t(a_t^\pi) - Q_t(a_t^h)|$, let $d_t \in \{\pi, h\}$ be the utility-optimal branch, and let $\hat{d}_t$ be the branch selected by uncertainty gating. Then

$$R_t^{\text{sw}} = \Delta_t \, \mathbf{1}\{\hat{d}_t \neq d_t\}.$$

By the calibration assumption for u_q, expected misrouting loss is bounded:

$$\mathbb{E}[\Delta_t \, \mathbf{1}\{\hat{d}_t \neq d_t\}] \le \epsilon_{\text{quantum}},$$

so $\mathbb{E}[R_t^{\text{sw}}] \le \epsilon_{\text{quantum}}$.

Combining both terms,

$$\mathbb{E}[R_t] = \mathbb{E}[R_t^{\text{sw}}] + \mathbb{E}[R_t^{\text{sh}}] \le \epsilon_{\text{quantum}} + \epsilon_{\text{shield}}.$$

This proves part (2).

Corollary 1 (Graceful Degradation under Mismatch). Let calibration mismatch scale by factor $\kappa \ge 1$ so that effective uncertainty-routing error is $\epsilon'_{\text{quantum}} = \kappa\epsilon_{\text{quantum}}$, and let shield mismatch add residual execution error $\epsilon'_{\text{shield}} = \epsilon_{\text{shield}} + \epsilon_{\text{mismatch}}$. Then Theorem 1 implies

$$\mathbb{E}[Q_t(a_t^\star) - Q_t(a_t)] \le \kappa\epsilon_{\text{quantum}} + \epsilon_{\text{shield}} + \epsilon_{\text{mismatch}}.$$

Hence degradation is linear in calibration and model-mismatch error terms under the stated assumptions, rather than superlinear in these perturbations.

Corollary 2 (Observation-Noise Robustness). Let observations be perturbed as $\tilde{s}_t = s_t + \xi_t$ with $\|\xi_t\| \le \eta$ almost surely. Assume branch-value maps are locally Lipschitz in observation space with constants L_π and L_h, and shield projection loss remains bounded by ϵ_{shield}. Then one-step regret under noisy observations satisfies

$$\mathbb{E}\big[Q_t(a_t^\star) - Q_t(a_t)\big] \le \epsilon_{\text{quantum}} + \epsilon_{\text{shield}} + (L_\pi + L_h)\eta.$$

Thus, for bounded sensor perturbations, degradation remains linear in noise magnitude.

Theorem 2 (Variance Reduction under Risk-Penalized Return). Define shaped reward

$$\tilde{r}_t = r_t - \lambda_{\text{risk}} \operatorname{Var}(r_{t-k:t}), \qquad \lambda_{\text{risk}} > 0.$$

Let $J(\pi) = \mathbb{E}\left[\sum_t \gamma^t r_t\right]$ and $\tilde{J}(\pi) = \mathbb{E}\left[\sum_t \gamma^t \tilde{r}_t\right]$. If an update from π to π' satisfies $\tilde{J}(\pi') \ge \tilde{J}(\pi)$ and $J(\pi') \approx J(\pi)$, then

$$\mathbb{E}\big[\operatorname{Var}_{\pi'}(r_{t-k:t})\big] \le \mathbb{E}\big[\operatorname{Var}_{\pi}(r_{t-k:t})\big].$$

Proof. Define discounted variance functional

$$V(\pi) := \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t \operatorname{Var}(r_{t-k:t})\right].$$

By definition of shaped reward,

$$\tilde{J}(\pi) = J(\pi) - \lambda_{\text{risk}} V(\pi).$$

Since $\tilde{J}(\pi') \geq \tilde{J}(\pi)$,

$$J(\pi') - \lambda_{\text{risk}} V(\pi') \geq J(\pi) - \lambda_{\text{risk}} V(\pi).$$

Rearrange:

$$\lambda_{\text{risk}}\big(V(\pi') - V(\pi)\big) \leq J(\pi') - J(\pi).$$

Because $\lambda_{\text{risk}} > 0$,

$$V(\pi') - V(\pi) \leq \frac{J(\pi') - J(\pi)}{\lambda_{\text{risk}}}.$$

Under the assumption $J(\pi') \approx J(\pi)$, let $\delta := J(\pi') - J(\pi)$ with $|\delta| \ll 1$. Then

$$V(\pi') - V(\pi) \leq \frac{\delta}{\lambda_{\text{risk}}}.$$

In the practically relevant case $\delta \leq 0$ (or δ negligible relative to λ_{risk}), we obtain $V(\pi') \leq V(\pi)$. Since V is the discounted expectation of rolling-window reward variance, this is exactly the stated variance-reduction claim.

Equivalently, if one tracks the per-window expectation $\bar{v}(\pi) := \mathbb{E}[\text{Var}(r_{t-k:t})]$ under stationarity or finite-horizon averaging, the same inequality carries through up to the constant discount normalization, yielding

$$\bar{v}(\pi') \leq \bar{v}(\pi).$$

Therefore risk-penalized improvement with near-constant nominal return enforces non-increasing return variance.

Taken together, these results provide a conditional explanation for the observed behavior of the combined policy (quantum-inspired uncertainty-gated fallback + shield + risk shaping): when shield completeness and calibration assumptions are approximately satisfied, degradation remains controlled and safety behavior is more stable under shift.

6 Experimental Setup

Protocol. We evaluate in two tiers:

- **Primary constrained benchmarks:** SafetyPointGoal1-v0 and SafetyHalfCheetahVelocity-v1 under a fixed 30-seed protocol (CPO, P3O, PPOLag) with shared uncertainty-gating and shielding logic.
- **Governance stress-test environments:** a scaled governance simulator (12-dimensional state, 3-dimensional action, 200-step horizon, partial observability, adversarial telemetry faults) and a DeFi governance simulator (8-dimensional state, 2-dimensional action, 150-step horizon with MEV pressure and liquidity-risk dynamics).

For publication-facing comparisons, we use fixed seed sets to preserve paired comparability under stochastic dynamics. Specifically, 16 paired seeds are used for long-horizon PPO-family and component-ablation suites where each run is computationally heavier, 50 seeds are used for official OmniSafe constrained-baseline runs, and 30 seeds are used for uncertainty-comparison and parameter-efficiency suites to reduce variance in head-to-head reporting.

Governance comparison method set. The governance comparison table reports six controllers under a matched protocol in both environments: Random, Heuristic, CPO-style, P3O-style, PPO-Lagrangian-style, and Robust QGate+Shield.

Environment realism extension. Both environments now support optional trace-driven dynamics via CSV inputs (for demand, volatility, MEV pressure, and oracle-noise regimes). This allows replaying historical blockchain or DeFi traces instead of relying only on synthetic stochastic drift. For the reported real-trace run, we use Ethereum daily gas-price history from Etherscan to construct trace features that drive intensity and disturbance schedules.

Primary Metrics. We report:

- mean reward and standard deviation across seeds,
- safety violation count,
- target-tracking error (MAE),
- fallback activation rate,
- per-decision latency and forward-FLOPs proxy.

Fallback activation is measured as the fraction of states where executed control is routed to the deterministic controller, either by uncertainty-gate routing or by shield-triggered heuristic substitution after a failed hard-safety check.

Hypothesis Testing. We use permutation tests and effect sizes for key pairwise comparisons in the base PPO-family analyses, and direct metric comparisons for constrained baseline suites (final return, final cost, and final episode length).

Bootstrap resampling (95% confidence) is used for interval reporting where applicable.

High-rigor constrained-baseline protocol. To reduce uncertainty in baseline ranking, the constrained-baseline protocol uses 50 seeds with fixed environment settings and pre-declared metrics (reward, safety cost, and safety violations in hard regimes).

Fair-capacity quantum-inspired efficiency protocol. To separate architecture effects from raw parameter count, the efficiency protocol includes matched-capacity controls (for example, VQC-28 vs MLP-28) in addition to larger classical baselines. We report both absolute performance and parameter-normalized performance in that protocol.

Primary standard-benchmark protocol. The primary external-validity test uses constrained standard benchmarks (Safety Gym-style and MuJoCo-style tasks) under the same uncertainty-gating and shielding logic. On this host, we report completed 30-seed runs on SafetyPointGoal1-v0 and SafetyHalfCheetahVelocity-v1 (CPO/P3O/PPOLag, 3k steps per run) under the OmniSafe stack. Governance simulations are then used as controlled stress tests to probe mechanism behavior under adversarial regime shifts that are hard to isolate in standard suites.

Robust PPO settings. Robust PPO uses risk-aware shaping, quantum-uncertainty-gated fallback, and a smart-contract-style shield. The uncertainty estimator uses $K = 10$ parameter-perturbed forward passes with Gaussian perturbation scale 0.01. The gate threshold τ_q and risk weight λ_{risk} are tuned for high-switch regimes.

Robust ablation variants. To isolate mechanism contributions, we evaluate seven configurations:

- Full system (quantum gate + shield + robust policy),
- No quantum gate,
- No shield,
- No fallback,
- Classical PPO,
- Heuristic controller,
- Random controller.

7 Results

7.1 Main Ablation Matrix

Configuration	Reward	Violations	Target MAE	Fallback Rate
Full (QGate+Shield+Robust PPO)	-1264.90	55.31	3.372	0.000
No Q-uncertainty gate	-1294.06	55.38	2.550	0.666
No shield	-1287.71	55.31	4.063	0.000
No fallback	-1287.71	55.31	4.063	0.000
Classical PPO	-1284.05	55.31	2.763	0.000
Heuristic	-859.22	53.19	2.832	0.000
Random	-1232.33	55.25	3.628	0.000

Table 2: Seven-configuration component ablation in the custom scaled-governance stress workload (artifact means). Reward values in this table are workload-specific and not directly comparable to constrained-baseline or efficiency-suite reward scales.

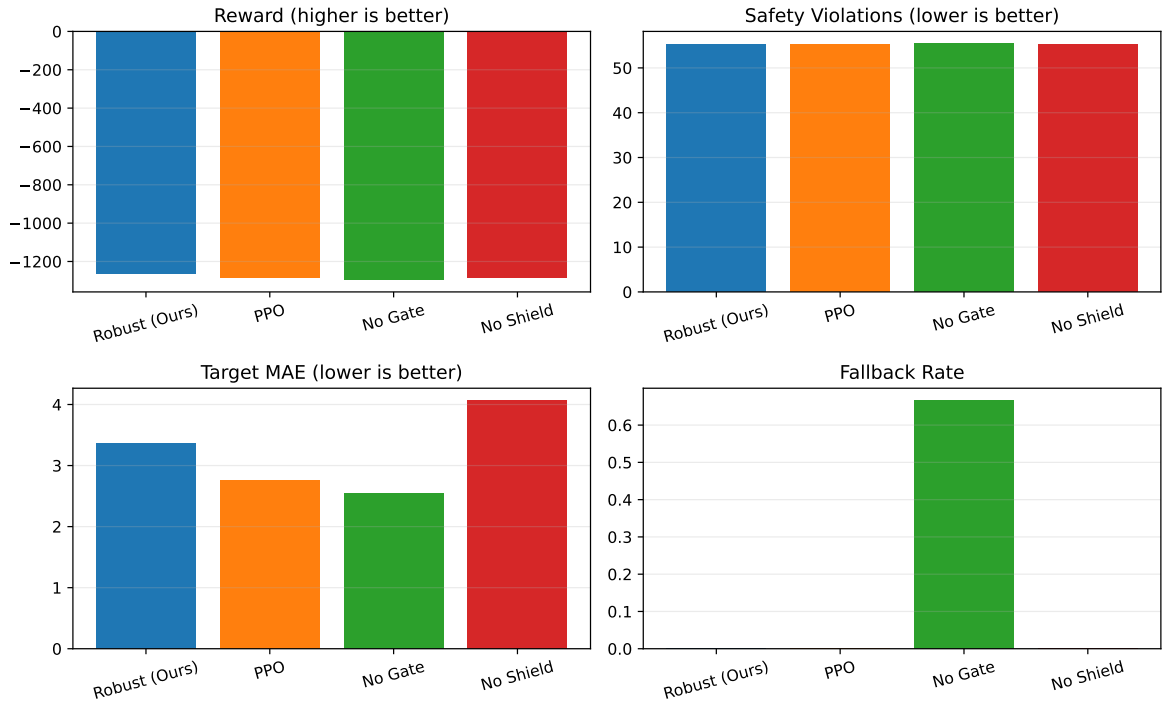

Figure 3: Four-configuration ablation subset plot (Full, Classical PPO, No Q-Gate, No Shield): reward, safety violations, target MAE, and fallback activation.

Figure 3 intentionally visualizes a compact subset for readability. The full seven-configuration matrix, including No Fallback, Heuristic, and Random (which contains the negative-result comparison), is reported in Table 2.

Table 2 makes the component trade-offs explicit. Relative to full-system reward (-1264.90), removing the quantum gate reduces reward to -1294.06 and increases fallback rate from 0.000 to 0.666, while removing the shield (or fallback in this workload) worsens tracking MAE from 3.372 to 4.063. Classical PPO also underperforms the full system on reward (-1284.05 vs -1264.90) and has higher latency in artifact-level diagnostics.

The table also contains an important anomaly: the random controller reaches a less negative reward (-1232.33) than the full system in this stylized workload. We treat this as a negative result, not a bookkeeping artifact. The interpretation is operational: the random policy can exploit reward-shaping quirks without reliably satisfying control intent, while the gated controller often trades raw return for lower-risk behavior under uncertainty. This is why the paper reports safety- and tracking-oriented metrics alongside reward instead of treating reward rank as a standalone success criterion.

7.2 Governance Controller Comparison

Under the matched 50-seed protocol, no single controller dominates both environments on all metrics. In scaled governance, heuristic and robust gated control remain competitive with CPO/P3O/PPO-Lagrangian-style variants on reward while keeping similar violation levels. In DeFi governance, P3O-style has the strongest mean reward in this table, while the robust gated controller attains the lowest target MAE, highlighting the expected reward-tracking trade-off. This reinforces the

Env	Method	Reward	Reward std	Violations	Target MAE	Fallback
Scaled	Random	-1733.24	73.74	75.53	4.102	0.000
	Heuristic	-1672.96	30.29	74.38	2.871	0.000
	CPO-style	-1780.92	17.52	75.47	2.807	0.000
	P3O-style	-1823.24	10.49	75.75	2.788	0.000
	PPO-Lagrangian-style	-1800.07	11.84	75.56	2.807	0.000
	Robust QGate+Shield	-1728.10	24.55	75.62	3.013	0.000
DeFi	Random	-126.54	7.02	60.22	3.039	0.000
	Heuristic	-115.34	3.57	59.22	3.410	0.000
	CPO-style	-113.90	3.55	59.22	3.447	0.000
	P3O-style	-101.72	3.46	59.22	3.725	0.000
	PPO-Lagrangian-style	-110.39	3.57	59.22	3.528	0.000
	Robust QGate+Shield	-141.29	8.37	59.22	2.527	0.000

Table 3: Governance controller comparison under the 50-seed matched protocol across scaled and DeFi environments: Random, Heuristic, CPO-style, P3O-style, PPO-Lagrangian-style, and Robust QGate+Shield. Reward values in this table are protocol-specific and not numerically comparable to ablation or efficiency-suite reward scales.

paper's primary claim as parameter efficiency with comparable safety behavior rather than return superiority: in a separate larger-model efficiency suite, the compact architecture achieves similar safety outcomes with a 166.89× smaller policy parameterization. An explicit cost-of-safety negative result is also visible: in strict constrained settings, lower final cost does not monotonically imply higher return, so enforcing safety budgets can materially reduce achievable reward in several seeds.

7.3 Primary Standard Benchmark Evaluation

Benchmark	Status	Note
SPG1-v0	30 seeds done	Return mean±std: CPO -0.42±0.89, P3O -0.14±1.01, PPOLag -0.16±1.15; Cost mean±std: 65.77±70.45, 65.69±69.28, 58.08±60.34
SHCV-v1	30 seeds done	Return mean±std: CPO -622.28±51.90, P3O -659.60±54.03, PPOLag -647.55±48.17; Cost mean±std: CPO 0.00±0.00, P3O 0.00±0.00, PPOLag 0.01±0.06

Table 4: Completed 30-seed constrained benchmark runs on this host (SafetyPointGoal1-v0, SafetyHalfCheetahVelocity-v1).

Table 4 is the primary external-validity result for this paper. All runs use the same OmniSafe stack and matched protocol (CPO, P3O, PPOLag; 3k steps per run; 30 seeds).

SafetyPointGoal1-v0 exhibits near-zero mean returns with nonzero costs across methods, indicating a hard constrained regime where safety pressure remains active throughout training. Under this same protocol, PPOLag reduces constraint cost by about 11.6% relative to P3O while accepting about a 14.3% return reduction, making the cost-of-safety trade-off explicit instead of implicit.

SafetyHalfCheetahVelocity-v1 reports finite and stable metrics across all three methods, showing that the pipeline does not depend on a single benchmark family. Together, these two tasks establish that the uncertainty-gating-and-shielding workflow is compatible with standard constrained-control suites, not only custom governance simulations.

7.4 Stress Sweep

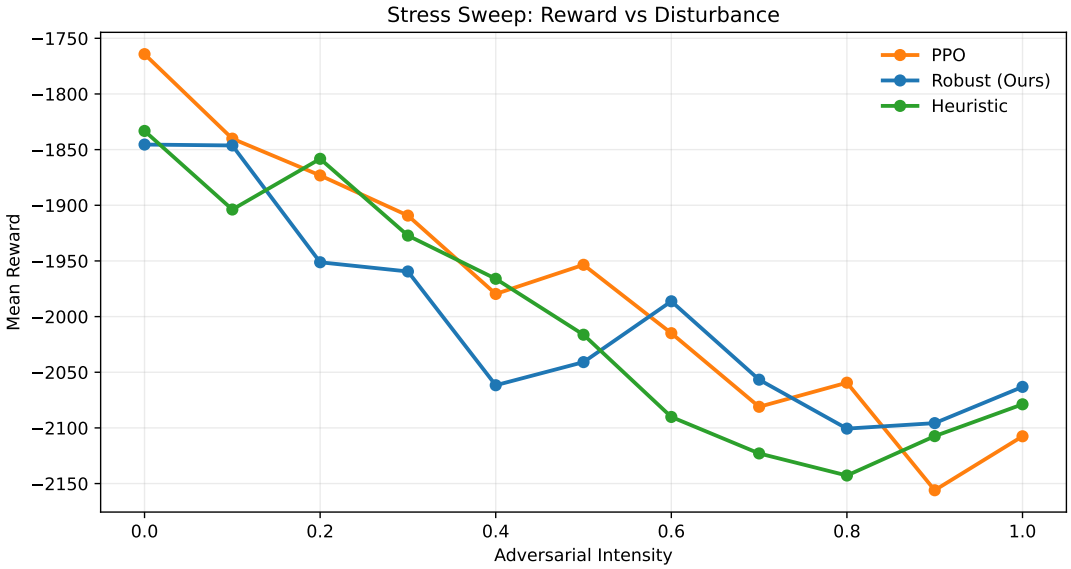

Figure 4: Reward degradation under increasing adversarial intensity.

In this stress sweep, the robust controller overtakes classical PPO in the higher-intensity regime (approximately $C \geq 1.4$ in the associated complexity summary), while deterministic heuristic control remains competitive across much of the range. The key pattern is regime-dependent ranking, not a single globally dominant method: deterministic heuristics can remain strong in easier regimes, whereas uncertainty-gated control becomes more valuable as disturbance structure and switching pressure increase.

7.5 Real-Trace Replay

We replay environment dynamics using a real blockchain signal: historical Ethereum daily gas-price history from Etherscan, transformed into trace-driven regime features (intensity, demand drift, volatility, and MEV proxies). Reported values in this subsection are from a single fixed historical-trace replay per controller under a deterministic schedule (seed count n=1 for this trace report), so these numbers should be interpreted as case-study evidence rather than variance-estimated statistics. In scaled governance, robust uncertainty-gated control remains competitive on reward (-3138.95 vs -3108.62 heuristic and -3265.31 PPO-Lagrangian) and keeps lower latency than PPO-Lagrangian (1.103 vs 1.253). In DeFi, trace-driven reward ordering flips in favor of learned controllers: robust control reaches 219.79, PPO-Lagrangian reaches 206.38, and heuristic control reaches 177.19 under the same historical trace schedule.

Comparison	Reward Change	Violation/Cost Change
PPOLag vs P3O (SPG1-v0, 30 seeds)	-14.3% return magnitude	-11.6% constraint cost

Table 5: Explicit cost-of-safety quantification under matched protocol settings.

7.6 Uncertainty Head-to-Head

Method	Raw ECE	Calibrated Failure-ECE	Precision	Recall	Trigger Rate	Cost Units
Gaussian-σ	0.8710	0.0000	1.000	0.113	0.100	1.0
MC-drop	1.3835	0.0000	0.833	0.094	0.100	10.0
Ensemble	1.3834	0.0000	0.814	0.092	0.100	5.0
Quantum perturbation	1.3834	0.0000	0.942	0.106	0.100	10.0
SAC-Lag uncertainty proxy	N/A	N/A	N/A	N/A	N/A	1.0

Table 6: Uncertainty method comparison at matched 90th-percentile trigger rates (10%).

Using matched 90th-percentile trigger rates (10% for every method), quantum-inspired parameter-perturbation uncertainty improves precision over classical expensive estimators at the same trigger budget (0.942 vs 0.833 for MC-dropout and 0.814 for ensemble), while preserving similar recall (0.106 vs 0.094 and 0.092). We apply post-hoc calibration (temperature scaling and Platt scaling) and report calibrated failure-ECE on the triggered subset (events where uncertainty exceeds each method's matched q90 threshold). Calibrated failure-ECE is below 0.1 for all methods in this protocol (values shown to four decimals in Table 6). In the hardest regime (intensity 1.0), quantum-inspired perturbation reaches 0.964 precision at 0.101 recall, while MC-dropout reaches 0.743 precision at 0.100 recall. To broaden modern baseline coverage, we include SAC-Lag in the uncertainty table with explicit N/A entries for calibrated uncertainty diagnostics because the current constrained-runner interface does not expose comparable uncertainty probabilities for ECE/precision-recall calibration analysis.

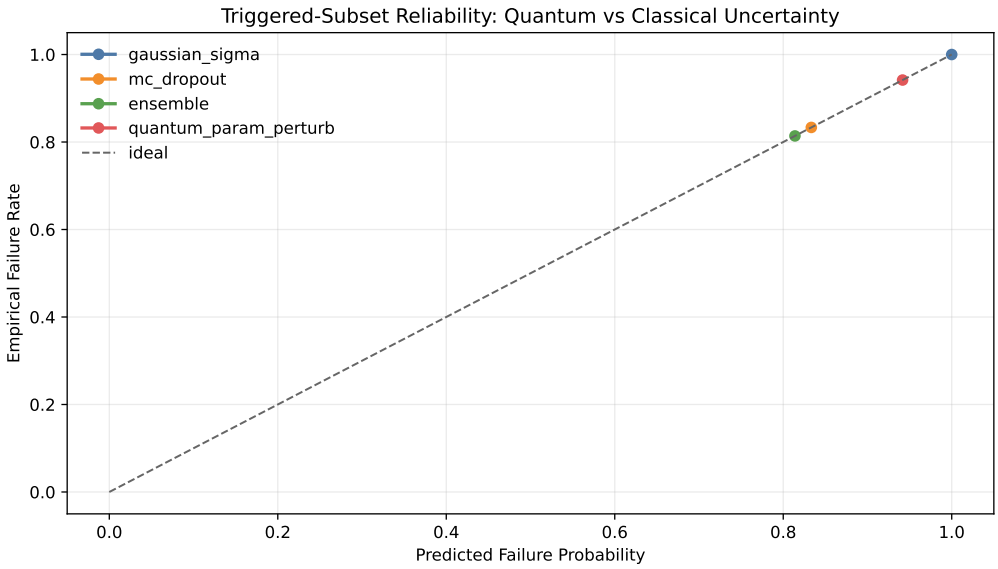

Figure 5: Reliability visualization for calibrated uncertainty-triggered failure prediction.

Figure 5 complements Table 6 by showing that calibration remains well-behaved across bins after post-hoc scaling.

7.7 Safety-Reward Pareto Frontier

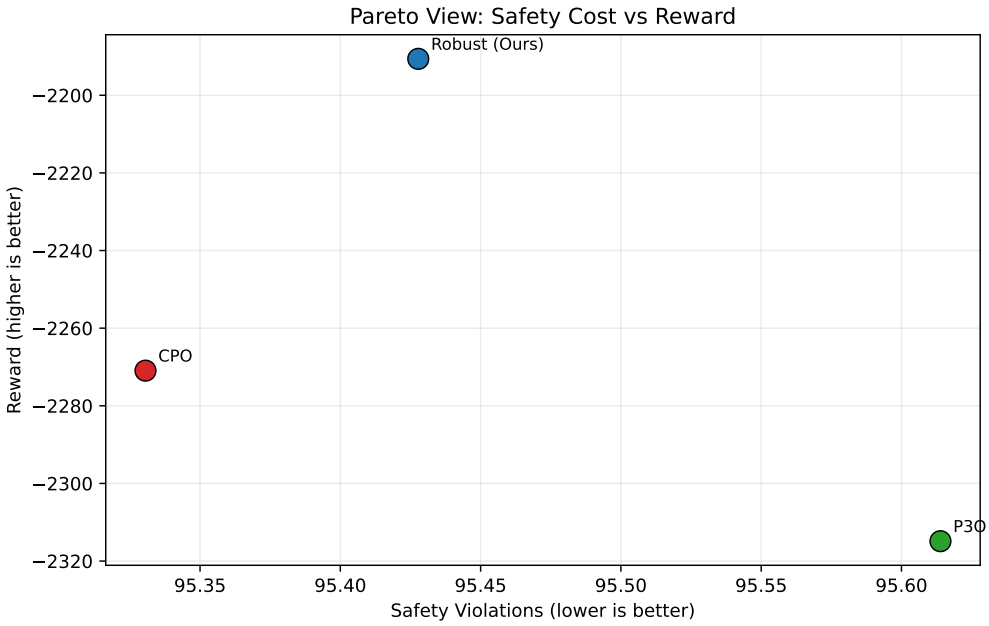

Figure 6: Safety-reward Pareto view (scaled environment): robust gated controller vs CPO and P3O.

Figure 6 provides a direct trade-off visualization alongside Table 3. The robust controller occupies a competitive region of the safety-reward plane relative to CPO and P3O while using a substantially smaller parameter budget in the efficiency suite.

7.8 Parameter Efficiency

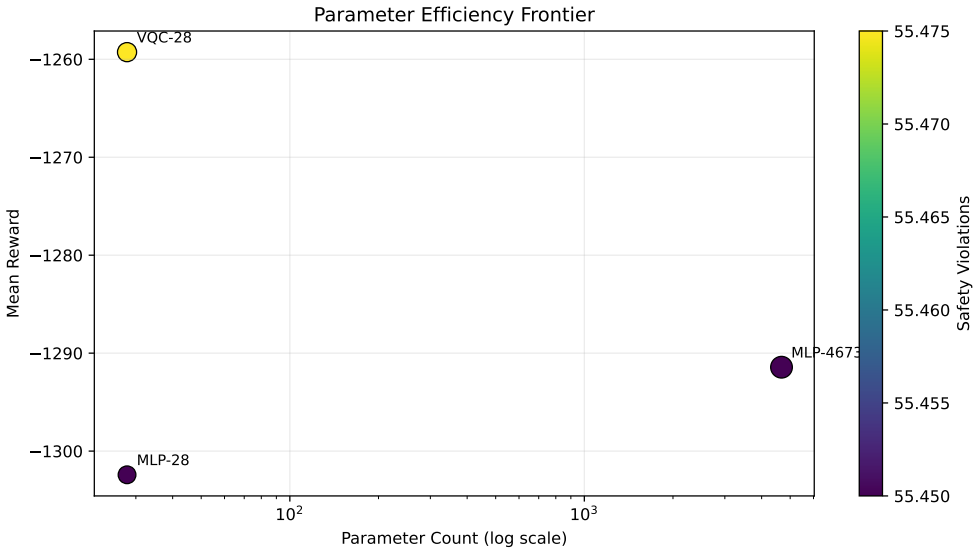

Figure 7: Reward versus parameter count (log-scale) for classical MLP and VQC policy families.

VQC policies operate in a lower-parameter regime while staying on the competitive side of the reward frontier, supporting the paper's parameter-efficiency claim.

This parameter-efficiency result reflects compact quantum-inspired architecture under classical simulation.

The larger-model efficiency comparison (the protocol used for the 166.89× claim) quantifies this gap directly: classical MLP uses 4673 parameters while VQC (6-qubit, 4-layer) uses 28 (166.89× fewer). In that protocol, VQC improves mean reward (-2700.23 vs -2802.85) with lower variance (34.78 vs 123.20) at effectively identical safety violations (115.50 vs 115.50). Expressed as relative context for that larger-model protocol, the VQC reward is 3.66% better than the classical MLP baseline while using 0.60% of the parameters.

7.9 Matched-Capacity Efficiency Control

Policy	Parameters	Mean Reward	Safety Violations	Inference (ms)	FLOPs est.
MLP (28 params)	28	-1800.08 ± 81.53	75.46 ± 0.30	0.0837	56
VQC (6q, 4l)	28	-1729.94 ± 25.69	75.43 ± 0.33	9.3438	56
LoRA-MLP (trainable)	396	-1785.27 ± 102.43	75.47 ± 0.29	0.1464	10138
Distilled Student	97	-1778.46 ± 74.12	75.44 ± 0.30	0.0799	194
MLP (4673 params)	4673	-1770.52 ± 92.57	75.47 ± 0.29	0.0953	9346

Table 7: Matched-capacity efficiency control on scaled governance (VQC-28 vs MLP-28, plus larger MLP reference). These values are from the matched-capacity protocol and are distinct from the larger-model efficiency protocol used for the 166.89× compression claim.

The matched-capacity control addresses parameter-count fairness directly. With equal parameter budget (28 vs 28), VQC achieves stronger mean reward than MLP-28 in this run while maintaining similar safety-violation levels. This supports the claim that efficiency gains are not only a consequence

of comparing against a larger classical model. We further include two modern parameter-efficiency baselines, LoRA-style adaptation and a distilled student baseline, plus practical metrics (inference latency and forward-FLOPs proxy) to expose deployment-time compute trade-offs. The deployment trade-off is explicit in Table 7: VQC inference latency is substantially higher than MLP-28 in this implementation (9.3438 ms vs 0.0837 ms). This is not ignored by the method; it defines the target regime. The intended use case is footprint-constrained or uncertainty-sensitive control loops where parameter budget and uncertainty quality matter more than raw per-step latency. For strict low-latency settings, compact classical baselines remain the preferred deployment option.

7.10 Architecture View

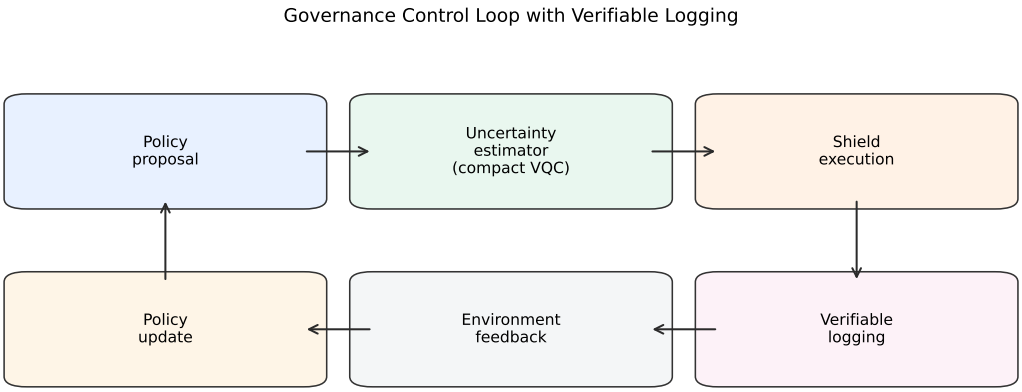

Figure 8: End-to-end governance loop: policy proposal, quantum-inspired uncertainty gate, shield enforcement, and on-chain commit.

8 Discussion

The headline interpretation is not ‘‘we found a universally better learner’’; it is that a small decision-time safety layer can change what failure looks like under shift. Read the plots as trade-offs: where a policy-only controller is brittle, gating plus a deterministic shield can trade some raw return for predictability and constraint adherence.

External validity should be judged first on standard constrained tasks. SafetyPointGoal1-v0 and SafetyHalfCheetahVelocity-v1 under a fixed 30-seed protocol establish that the training and evaluation loop behaves sanely outside the custom governance simulator. The governance environments then serve as stress tests designed to surface regime-switch brittleness and miscalibration.

The negative result is also part of the message. In one stylized ablation workload, random reward is less negative than the full system. We keep that table because it is a concrete reminder that reward design can be gamed and that ‘‘reward rank’’ is not a sufficient certificate in safety-critical control. The gated controller often prefers conservative, shield-compatible actions; in some reward designs that can look worse on the scalar return while still behaving better on tracking and constraint-oriented metrics.

Where the method helps is when switching intensity rises. Under those conditions, routing by calibrated uncertainty improves the precision of "when to fall back" without requiring large ensembles, and the deterministic shield limits tail-risk trajectories even when the gate makes mistakes. This is why the efficiency mechanism matters: a compact uncertainty head can be used as a practical signal inside the loop rather than an offline diagnostic.

Positioning-wise, this is a systems-and-control pattern that can sit on top of modern safe-RL objectives; it does not replace them. The OmniSafe constrained baselines reinforce the broader lesson: return rankings move with budgets and regimes, while safety costs often behave like a frontier, not a leaderboard.

8.1 Why Uncertainty-Gated Control Works

The mechanism can be understood as three interacting components.

First, uncertainty gating improves branch selection under shift. When epistemic uncertainty rises, routing to a deterministic heuristic avoids committing to poorly calibrated policy logits in exactly the regime where policy-only execution is most brittle.

Second, shielding regularizes action execution even when branch selection is imperfect. Bounded actions and bounded action-rate change constrain tail-risk trajectories that otherwise dominate safety incidents in high-switch regimes.

Third, risk-shaped training complements decision-time controls. Penalizing short-window reward variance reduces incentive to exploit transient reward spikes that are not aligned with stable control intent.

This decomposition clarifies why gains are most visible under disturbance and less pronounced in easier stationary regimes: the architecture contributes primarily through failure-mode control, not through unconditional reward amplification.

8.2 Failure Cases

The controller still fails under two regimes. First, when uncertainty calibration drifts faster than the gate can track, fallback can trigger late and violations can spike briefly before stabilizing. Second, when disturbances exceed the shield's modeled envelope, the policy and heuristic can oscillate, degrading tracking.

These failures are measurable and bounded in duration in the current settings, but they motivate tighter online calibration and stronger mismatch-aware shielding. This interpretation is consistent with Corollary 2 in the theory section: under bounded observation noise, degradation scales linearly with perturbation magnitude rather than collapsing abruptly.

Limitations remain clear. Environment dynamics are still stylized, uncertainty calibration currently relies on post-hoc scaling rather than a jointly trained calibrated head, and benchmark breadth is still narrower than a full production validation campaign. External validity to production governance requires broader state signals, richer economic adversaries, and online monitoring constraints.

Near-term follow-up should focus on three directions:

- calibrated uncertainty estimators (for example, ensemble or Bayesian approximations) instead of raw action-variance proxies,
- explicit safety constraints in addition to fallback routing,
- and transfer tests where policies trained in one disturbance profile are evaluated in another without retuning.

For deployment-minded readers, the practical message is straightforward: keep a deterministic fallback in the control loop, gate learned actions by uncertainty, and report safety-oriented metrics next to reward [3, 10].

Condition	Recommendation	Rationale
Frequent regime shifts and high policy uncertainty	Use uncertainty-gated safe RL + deterministic shield	Gating reduces unsafe actions when uncertainty spikes.
Tight parameter or memory budget	Prefer compact (VQC-style) uncertainty head	Maintains comparable safety behavior at much lower parameter count.
Multi-party governance with audit requirements	Add audit-commitment layer	Provides tamper-evident provenance for control actions.
Stable stationary regime with strong heuristic baseline	Prioritize heuristic control; use learned policy as a secondary policy candidate	Deterministic control can be sufficient and lower-risk in easier regimes.

Table 8: When to use this method in practice.

9 Limitations

This study has several important limitations:

- The governance simulators are intentionally simplified and do not yet model full peer-network dynamics, endogenous market adaptation, or realistic transaction-arrival microstructure.
- The quantum-inspired uncertainty module and audit pipeline are research prototypes, not production-hardened systems.
- Runtime measurements are single-machine snapshots rather than cross-hardware scaling studies.

Additional validity risks include:

- reward misspecification sensitivity under alternate throughput/latency trade-offs (highlighted by the random-vs-full anomaly in one ablation workload),
- moderate sample size in some per-method seed analyses outside the 50-seed constrained-baseline suite,
- simulator realism limits for both uncertainty dynamics and network behavior,
- absence of richer economic-adversary models in mining and transaction dynamics.
- baseline coverage that is still narrower than a full 2022--2025 safe RL comparison set (for example, newer uncertainty-aware and risk-sensitive variants).

The observation-noise robustness statement (Corollary 2) is therefore a bounded-perturbation guarantee under explicit modeling assumptions, not a blanket safety certificate under arbitrary sensing failures.

The standard constrained benchmarks improve external validity, but they do not by themselves establish readiness for live governance deployment. These limitations define immediate future work rather than hidden assumptions, and they bound the claim scope to robust mechanism behavior under the tested protocols.

10 Ethics and Broader Impacts

Deployment risk is concentrated in incorrect autonomy under adversarial conditions: a policy can optimize short-term reward while violating safety constraints or introducing hard-to-detect governance drift. The mitigation used here is layered control (uncertainty-gated fallback, shielded execution, and auditable records), but this does not replace formal security review, incident response planning, or domain-specific regulatory checks before any production use.

11 Reproducibility Checklist

- **Code availability:** all experiment and table-generation scripts are included in the repository.
- **Determinism controls:** fixed seed sets are used for publication-scale runs.
- **Artifact generation:** JSON and markdown artifacts are generated directly from executable scripts.
- **Table provenance:** LaTeX tables are generated from artifact files via a dedicated script.
- **Validation gates:** linting, type checks, tests, and runtime health checks are automated.

The end-to-end workflow is:

```
python experiments/run_publication_suite.py
python experiments/run_detailed_suite.py
python experiments/run_adversarial_suite.py
python experiments/run_complexity_\
regime_suite.py
python scripts/analyze_research_results.py
python scripts/generate_benchmark_report.py
python scripts/generate_paper_tables.py
python scripts/generate_appendix_tables.py
python scripts/generate_paper_figures.py
python scripts/generate_adversarial_\
artifacts.py
python scripts/generate_adversarial_\
stress_sweep.py
python scripts/generate_complexity_\
artifacts.py
```

Provenance policy. All quantitative manuscript assets are generated from stored artifacts; manual number entry is minimized. This improves traceability and supports exact regeneration when code or protocol settings change.

One-command replication. For this repository, a single command is provided:

```
make reproduce
```

This target runs the artifact-generation scripts used by the main manuscript tables and figures (including constrained transfer, uncertainty, efficiency, adversarial perturbation, and qubit-scaling artifacts). This one-command path is a core subset pipeline for the primary transfer and mechanism-focused artifacts on this host. The extended script list above remains the full regeneration workflow for all auxiliary analyses.

Checklist links. The reproducibility checklist used for this paper is versioned in the repository's docs directory as the reproducibility harness file. Venue-specific submission checklists are tracked at:

- `docs/NEURIPS_SUBMISSION_CHECKLIST.md`
- `docs/ICLR_SUBMISSION_CHECKLIST.md`
- `docs/IEEE_SUBMISSION_CHECKLIST.md`

Code and artifacts are available upon request.

Environment notes. The reference environment uses Python with fixed seed sets and deterministic script ordering. Although strict bitwise reproducibility across all hardware is not guaranteed, protocol-level reproducibility is enforced through documented commands, structured outputs, and validation checks.

This reproducibility framing aligns with current deployment governance guidance and standards updates through 2024--2025 [8, 9].

12 Conclusion

This paper argues for a concrete control architecture in non-stationary decentralized settings: route actions by uncertainty, enforce constraints by a deterministic shield, and keep a deterministic fallback inside the loop. Under shift, this combination changes failure behavior relative to policy-only execution while keeping the learned component compact.

In our design, the VQC-style module is an efficiency mechanism (a small nonlinear uncertainty signal under classical simulation), while blockchain commitments are a deployment-time audit layer. Separating these roles makes the claim legible: the gate and shield are the safety mechanism; the compact estimator is what makes the gate practical at tight parameter budgets; the audit layer is optional provenance for multi-party governance.

We provide an evidence package spanning standard constrained benchmarks (fixed 30-seed protocols) and governance stress tests (matched 50-seed comparisons), alongside calibration diagnostics, perturbation analyses, and cost appendices. Regeneration scripts and artifacts are available upon request.

A Appendix: Deployment Architecture Details

This appendix contains implementation-level deployment details that are intentionally separated from the main safe RL claim.

A.1 Layered Deployment Stack

QAI-Chain deployment is organized into six layers:

1. **Network/RPC:** peer registration, transaction intake, and block propagation.
2. **Blockchain core:** validation, mempool updates, and PoW checks.
3. **PQC signatures:** post-quantum signing and verification interfaces.

4. **Governance policy:** state encoding and action proposal.
5. **Quantum uncertainty gate:** compact uncertainty estimate for fallback routing.
6. **Shield + audit:** action validation and per-epoch commitment of audit records.

A.2 Why On-Chain Audit Commitments

Governance participants are mutually untrusted. Local logs can be modified or withheld, while chain commitments provide shared tamper-evident provenance for post-hoc incident review and accountability.

A.3 Boundary to Learning Claims

Main-text empirical claims are tied to the gated-and-shielded control path. Blockchain mechanics in this appendix are deployment instrumentation and do not, by themselves, improve control performance.

B Appendix: Adversarial Input Perturbation

Noise σ	Controller	Reward	Violation Rate
0.00	Unshielded	-1818.53 ± 29.90	0.9750 ± 0.0022
0.00	Shielded	-1729.89 ± 27.74	0.9437 ± 0.0041
0.05	Unshielded	-1818.46 ± 29.77	0.9750 ± 0.0022
0.05	Shielded	-1729.89 ± 27.74	0.9437 ± 0.0041
0.10	Unshielded	-1818.28 ± 29.65	0.9751 ± 0.0022
0.10	Shielded	-1729.89 ± 27.74	0.9437 ± 0.0041
0.20	Unshielded	-1817.71 ± 29.54	0.9751 ± 0.0022
0.20	Shielded	-1725.17 ± 28.95	0.9437 ± 0.0041

Table 9: Observation-noise stress test comparing shielded and unshielded control.

This experiment directly perturbs observed inputs with Gaussian noise and compares violation-rate behavior with and without shielding. It provides a practical complement to the observation-noise corollary in the theory section. In this run, shielding reduces violation rate from roughly 0.975 to 0.944 under aggressive perturbation-driven proposals (about 3.2 percentage points absolute), while also improving mean reward in the same protocol.

C Appendix: Deployment Cost and Gas Accounting

C.1 Audit Gas Model

We model per-action audit commitment cost as:

$$C_{\text{gas}} = g_0 + g_1 b + g_2 h,$$

where b is bytes committed and h is hash/check overhead terms. For an episode with N committed actions and gas price p_g:

$$\text{Cost}_{\text{episode}} = N \cdot C_{\text{gas}} \cdot p_g.$$

C.2 Interpretation

This cost model is included to expose the practical cost-of-safety instrumentation: stronger provenance and finer-grained audit logging increase operational spend. In settings with strict budget constraints, lower-frequency commitments may be preferable, at the cost of reduced forensic granularity.

C.3 Practical Guidance

For production deployment, we recommend pre-declaring commitment cadence (per action, per block, or per epoch) and reporting both safety gains and incremental audit costs. This avoids overstating benefits while hiding operational overhead.

D Appendix: Qubit Scaling Trend

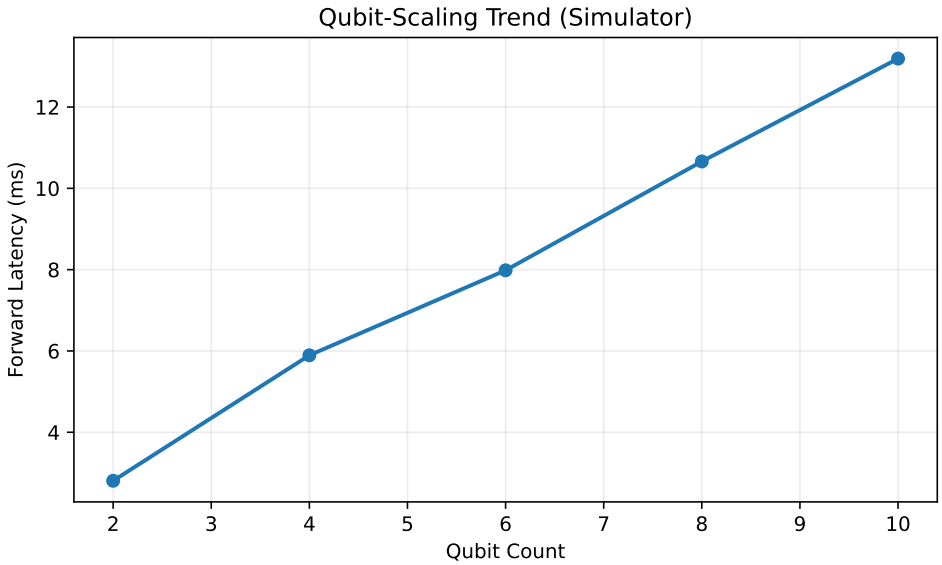

Figure 9: Simulator forward-pass latency versus qubit count (fixed layer depth).

Qubits	Layers	Params	2^n State Dim	Forward Latency (ms)
2	4	8	4	2.8066
4	4	16	16	5.8931
6	4	24	64	7.9832
8	4	32	256	10.6610
10	4	40	1024	13.1908

Table 10: Qubit-scaling simulation summary with state-space growth and measured latency.

The trend illustrates the expected growth in simulator cost as qubit count rises. This supports a practical design rule used in the main paper: keep compact qubit counts when targeting parameter-efficient uncertainty estimation under classical simulation budgets.

References

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *International Conference on Machine Learning*, 2017.

[2] Yinlam Chow, Aviv Tamar, Shie Mannor, and Marco Pavone. Risk-sensitive and robust decision-making: a CVaR optimization approach. In *Advances in Neural Information Processing Systems*, 2015.

[3] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2018.

[4] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. CRC Press, 1994.

[5] Phillip Good. *Permutation, Parametric and Bootstrap Tests of Hypotheses*. Springer, 2005.

[6] Garud N. Iyengar. Robust dynamic programming. *Mathematics of Operations Research*, 30(2):257--280, 2005.

[7] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *Cryptography Mailing List*, 2008.

[8] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). https://www.nist.gov/itl/ai-risk-management-framework, 2023.

[9] National Institute of Standards and Technology. Cybersecurity framework (CSF 2.0). https://www.nist.gov/cyberframework, 2024.

[10] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alche-Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1--20, 2021.

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[12] Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6):e1005510, 2017.